

Investigating the Scalability of Approximate Sparse Retrieval Algorithms to Massive Datasets

Sebastian Bruch^{1*}[0000–0002–2469–8242],
Franco Maria Nardini^{2*}[0000–0003–3183–334X],
Cosimo Rulli^{2*}✉[0000–0003–0194–361X],
Rossano Venturini^{3*}[0000–0002–9830–3936], and
Leonardo Venuta^{4*}

¹ Northeastern University, Boston, USA, s.bruch@northeastern.edu

² ISTI-CNR, Pisa, Italy, {name.surname}@isti.cnr.it

³ University of Pisa, Italy, rossano.venturini@unipi.it

⁴ University of Pisa, Italy, l.venuta@studenti.unipi.it

Abstract. Learned sparse text embeddings have gained popularity due to their effectiveness in top- k retrieval and inherent interpretability. Their distributional idiosyncrasies, however, have long hindered their use in real-world retrieval systems. That changed with the recent development of approximate algorithms that leverage the distributional properties of sparse embeddings to speed up retrieval. Nonetheless, in much of the existing literature, evaluation has been limited to datasets with only a few million documents such as MsMARCO. It remains unclear how these systems behave on much larger datasets and what challenges lurk in larger scales. To bridge that gap, we investigate the behavior of state-of-the-art retrieval algorithms on massive datasets. We compare and contrast the recently-proposed SEISMIC and graph-based solutions adapted from dense retrieval. We extensively evaluate SPLADE embeddings of 138M passages from MsMARCO-v2 and report indexing time and other efficiency and effectiveness metrics.

Keywords: Sparse Embeddings · Approximate Large-scale Retrieval.

1 Introduction

Sparse embeddings [19,13,12,10,16] have proven effective in capturing the contextual semantics of text. Such embeddings have matured over the past few years and found their way into first-stage retrieval [3].

Neural models that are trained to produce sparse embeddings rewire a Large Language Model (LLM) to generate a high-dimensional vector for an input text. Each coordinate of the output represents a term from the model’s vocabulary, and if it is nonzero, its corresponding term is taken to be *semantically* relevant to the input. Similarity between embeddings is by inner product, making top- k retrieval an instance of Maximum Inner Product Search (MIPS).

* All authors contributed equally to this work.

Sparse embeddings have three useful properties. First, repeated experiments show that they are as effective as *dense embeddings* [18,14,31,26,27,15,24] and generalize better to out-of-domain datasets [2,16,29]. Second, the one-to-one mapping between dimensions and terms makes them inherently *interpretable*. Third, they fit naturally in the inverted index-centric paradigm [30], while remedying the *vocabulary mismatch* problem at the same time.

Unfortunately, a naïve application of inverted index-based retrieval to sparse embeddings does not satisfy tight latency constraints of real-world systems. That stems from distributional differences between embeddings and bag-of-words representations of text [4,20]. That challenge gave rise to works that attempt to speed up (safe or unsafe) top- k retrieval for sparse embeddings [4,5,17,23,21].

Among these methods, the approximate (unsafe) retrieval method by Bruch *et al.* [6] stands out. The algorithm they call SEISMIC uses an inverted index as usual, but organizes its inverted lists into geometric blocks—similar in spirit to [8,1]. Each of these blocks is then represented with a *summary* vector. During query processing, SEISMIC judges if a block must be evaluated based on whether or not its summary has a “high-enough” inner product with the query.

A comprehensive empirical evaluation of SEISMIC on sparse embeddings of MSMARCO [25] showed that it reaches sub-millisecond per-query latency with high recall. That is one to two orders of magnitude faster than inverted index-based solutions and more efficient than graph-based methods from the 2023 BIGANN Challenge [28]. Later, the same authors improved SEISMIC further, making it almost exact but up to $2.2\times$ faster than the original SEISMIC [7].

In all the works discussed above, the retrieval algorithms have been evaluated only on datasets no larger than MSMARCO, which is made up of roughly 8.8 million embedding vectors. What happens if we applied these methods to a much larger dataset, such as MSMARCO v2, with about 138 million embedding vectors? What surprising challenges lurk in such scales that do not ordinarily surface? How long will indexing take and how much memory will it require? Does retrieval accuracy deteriorate as the collection grows in size?

We investigate these questions empirically by subjecting approximate retrieval algorithms designed for learned sparse embeddings to this massive dataset. To the best of our knowledge, no work has yet studied the efficiency and effectiveness of such methods at the extreme scale we consider in this work.

2 Experimental Setup

In this study, we compare two main families of retrieval algorithms over sparse embeddings of massive datasets: graph-based and inverted index-based.

Graph-based algorithms. The 2023 BIGANN Challenge [28] organized a track for sparse vector retrieval over SPLADE [11] embeddings of MSMARCO [25]. Two graph methods emerged as clear winners: PYANN and GRASSRMA.

PYANN builds a graph index with HNSW [22], but uses a modified search algorithm during query processing. In particular, it quantizes vectors so that coordinates are represented as 16-bit integers, and values as 16-bit half-precision

floats. Furthermore, it discards smaller values of query vectors. The retrieved set is reranked in a refinement step that uses full vectors to compute inner products.

GRASSRMA also uses HNSW for graph index construction. Similar to PYANN, it introduces a few minor optimizations: it co-locates coordinates and values to improve memory access, and employs an upper- and lower-bound of the values per vector to terminate inner product computation early.

Recently, a new implementation of HNSW has been made available in the KANNOLO [9] library,⁵ a framework in Rust for approximate nearest neighbors search targeting both dense and sparse domains. We choose this solution because, as Delfino *et al.* show, it outperforms both GRASSRMA and PYANN in terms of retrieval time [9].

SEISMIC. In contrast to the methods above, SEISMIC operates on the inverted and the forward index. The crucial innovation is that, each inverted list is clustered into geometric blocks, each equipped with a *summary* vector. An example summary would be a vector that records the maximum of each nonzero coordinate among vectors in that block. See [6] for the full description.

SEISMIC executes a term-at-a-time strategy to produce the top- k results for a query q . When traversing an inverted list, it first computes the inner product between q and every summary in that list to compute a “potential” score. It then visits each block, comparing its potential with the smallest score in the top- k heap. If a block’s potential exceeds that threshold, SEISMIC uses the forward index to compute the exact inner product between q and every document in that block. This allows the query processor to skip over a large number of blocks.

A follow-up work [7] adds a κ -NN graph to the index: a graph where vectors are nodes, and each node is connected to its κ nearest neighbors by inner product. The updated algorithm uses the κ -NN graph as follows. Once the retrieval procedure described above concludes its search, it takes the set of k documents in the heap, denoted by $\mathcal{S}$. It then forms the *expanded set* $\tilde{\mathcal{S}} = \bigcup_{u \in \mathcal{S}} (\{u\} \cup \mathcal{N}(u))$, where $\mathcal{N}(u)$ is the set of neighbors of u in the κ -NN graph; computes scores for documents in $\tilde{\mathcal{S}}$ using the forward index; and returns the top- k subset. The SEISMIC code is available on GitHub.⁶

Dataset. We conduct our experiments on SPLADE [13] embeddings of MS-MARCO v2, a collection of 138 million passages with 3,903 queries in the `dev1` set.⁷ Retrieval over these embeddings yields MRR@10 of 10.88%, MRR@100 of 11.74%, and Recall@1000 of 61.87%.⁸ Passages (queries) have 127 (44) nonzero entries each, which is close to the statistics of SPLADE on MSMARCO: 119 (43).

Index size budget. We allocate memory budgets for the index as multiples of the dataset size, which is about 66GB with 16-bit floats for values and 16-bit integers for components. We only consider hyperparameters that result in indexes that are up to $1.5\times$ the dataset size in one scenario, and $2\times$ in another.

⁵ <https://github.com/TusKANNy/kannolo>

⁶ <https://github.com/TusKANNy/seismic>

⁷ `cocondenser-ensembledistil` version from <https://github.com/naver/splade>.

⁸ The embeddings of MSMARCO v2 used in this study are made available at <https://huggingface.co/collections/tuskanny>.

Hyperparameters. The graph index has the following hyperparameters: M (number of neighbors per node), and ef_c (number of nodes scanned to build a node’s neighborhood). We let $M \in \{16, 32, 64\}$ and $ef_c = 500$. For SEISMIC, we set: number of postings, $\lambda \in \{3, 4, 5, 6, 7, 8, 9\} \times 10^4$; summary energy, $\alpha = 0.4$; and number of blocks per list, $\beta = \lambda/10$. These are the best reported values on MsMARCO. We let κ to take values in $\{10, 20\}$, and build an (approximate) κ -NN graph by using a SEISMIC index with $\lambda = 6 \times 10^4$, cut set to 20, and heap_factor to 0.6.

We pick the best configuration following the same protocol as [6,7,28]. We reiterate that, if our budget is $1.5\times$ the dataset size, we limit our search to hyperparameters that produce an index that is smaller than that budget.

Given an index, we process queries using the following hyperparameters and evaluate each configuration separately. For SEISMIC, we vary cut in $\{2, \dots, 14\}$, with step 2, heap_factor in $\{0.6, \dots, 1.0\}$, and κ in $\{10, 20\}$. For the graph index, we vary ef_s in $[10, 50]$ with step 5, $\{50, 100\}$ with step 10, $\{100, 1500\}$ with step 100. For both methods, we consider the shortest time it takes the algorithm to reach accuracy@10 for values in $\{90, \dots, 98\}$.

Metrics. We evaluate along the following axes:

- Average Latency (milliseconds, *msec*): Elapsed time to retrieve top- k vectors for a query with a single thread. This does not include embedding time.
- Accuracy: Percentage of true nearest neighbors recalled.
- Index size (GB): The space the index occupies in memory.
- Indexing time: The time required to build the index with all available threads.

Hardware Details. We use version 1.81 of the Rust compiler and compile using the `release` option. We run experiments on a NUMA server with 1TiB of RAM and four Intel Xeon Gold 6252N CPUs (2.30 GHz), with a total of 192 cores (96 physical and 96 hyper-threaded). Using the `numactl`⁹ tool, we enforce that the execution of the retrieval algorithms is done on a single CPU and its local memory. In this way, we avoid performance degradation due to non-uniform memory accesses across different CPUs. We note that, each CPU node has enough local memory (256GiB) to store entire indexes. We leave the study of cases where the index does not fit in the main memory to future work.

3 Experimental Results

Our first experiments, visualized in Figure 1, test HNSW as implemented in KANNOLO and SEISMIC by accuracy@10 as a function of query latency. SEISMIC remains Pareto-efficient relative to HNSW in both memory budget settings.

Interestingly, when we allow the algorithms to use more memory (i.e., $2\times$ the dataset size), the gap between SEISMIC and HNSW widens as accuracy increases. That is thanks to the κ -NN graph, which gives an advantage in high accuracy scenarios ($\geq 97\%$), as validated by a comparison of SEISMIC with and without

⁹ <https://linux.die.net/man/8/numactl>

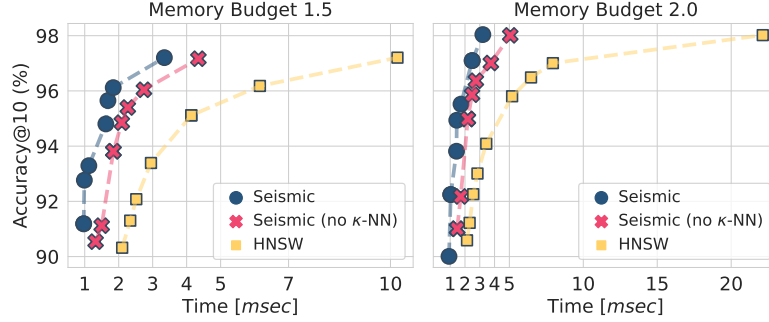


Fig. 1. Comparison of HNSW and SEISMIC (with and without κ -NN graph) by accuracy at $k = 10$ as a function of query latency. We allow hyperparameters that result in an index whose size is at most $1.5\times$ (left) or $2\times$ (right) the size of the dataset.

the κ -NN graph. As an example, in the $1.5\times$ scenario, SEISMIC with (without) κ -NN is $3.0\times$ ($2.3\times$) faster than HNSW at 97% accuracy cutoff. The speedups increase to $6.9\times$ ($4.4\times$) in the $2\times$ scenario at 98% accuracy cutoff.

Observe that the κ -NN graph has a notable impact on memory: Storing it costs $(\lceil \log_2(n-1) \rceil + 1)n\kappa$ bits, with n denoting the size of the collection, which translates to 27 bits per document in our setup. This analysis suggests that reducing the memory impact of the κ -NN graph, for example, using delta encoding to store the ids of the documents, may help SEISMIC become even faster at a given memory budget.

Index construction time and size. SEISMIC builds the MsMARCO index much more quickly than HNSW (c.f., Table 2 in Bruch *et al.* [6]). This advantage holds strong on MsMARCO v2, as we show in Table 1: SEISMIC (without the κ -NN graph) builds its index in about 30 minutes, while HNSW in the sparse domain takes about 12.5 hours. The reported configuration refers to the $1.5\times$ memory budget scenario.

Enabling the κ -NN graph in SEISMIC substantially increases the indexing time. That is to be expected: Every document in the collection becomes a query. Despite the construction of the κ -NN graph being approximated with SEISMIC, this process involves the daunting task of searching through approximately 140 million queries. Moreover, as the collection size grows, not only would one have more queries to search, but there would be a larger number of documents to search over. As such, the inference benefits of the κ -NN graph shown in Figure 1 come at a high cost at indexing time.

3.1 Scaling Laws

In Figure 2, we present the retrieval time scaling laws for SEISMIC and HNSW. For each accuracy threshold on the horizontal axis, we plot on the vertical axis the ratio of the search time on MsMARCO v2 to search time on MsMARCO.

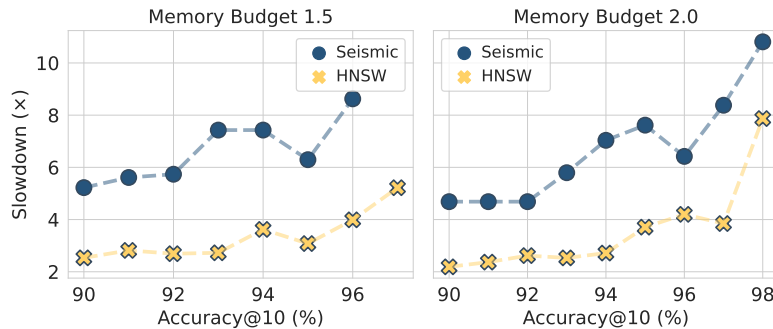


Fig. 2. Scaling laws of SEISMIC and sparse HNSW (as provided by the KANNOLO library). For each accuracy cutoff, we measure the ratio between the latency of a method on MSMARCO v2 and on MSMARCO.

In doing so, we report the amount of slowdown between the two datasets. In computing these ratios, we use the same hyperparameters as Bruch *et al.* [7] for MSMARCO; for MSMARCO v2, we use the hyperparameters noted earlier that gave the results in Figure 1.

Considering the fact that MSMARCO v2 is approximately $15\times$ larger than MSMARCO, from Figure 2 we learn that both methods scale effectively with dataset size. Concerning SEISMIC, all configurations show a slowdown of less than $8\times$, except for accuracy thresholds of 97 and 98. Regarding HNSW, in the $1.5\times$ memory budget scenario, slowdowns are reduced, ranging from $2.5\times$ to $5.2\times$. In the $2\times$ memory budget, the slowdown is small for mid accuracy cuts (90-94) and then, as for SEISMIC, it increases reaching a peak of $7.9\times$ at 98.

4 Conclusions and Future Work

We investigated the performance of state-of-the-art approximate sparse retrieval algorithms on SPLADE embeddings of MSMARCO v2. We focused our analysis to the recently-proposed SEISMIC and graph-based algorithms.

Our analysis confirms what has been known anecdotally: Building a graph index for approximate nearest neighbor search is a resource-intensive and time-

Table 1. Index size and build time for winning configurations.

Model	Configuration	Index size (GB)	Build time (hours)
HNSW	M=32	104.4	10.1
SEISMIC	$\lambda = 4e4, \kappa = 20$	98.4	16.6
SEISMIC (no κ -NN)	$\lambda = 6e4$	98.5	0.5

consuming effort. Building an HNSW index (using an efficient implementation from KANNOLO) takes $25\times$ more time than SEISMIC.

Our experiments also reveal that SEISMIC processes queries with a much lower latency than a graph method, and that the gap between the two methods widens as we demand a higher retrieval accuracy. For example, at 98% accuracy@10 in the $2\times$ budget setting, SEISMIC returns results in 3 milliseconds per query, whereas sparse HNSW is $6.9\times$ slower. It is, however, worth noting that SEISMIC’s query latency scales less favorably than HNSW.

As future work, we intend to investigate efficient retrieval over massive datasets addressing two specific scenarios: 1) efficient retrieval where indexes cannot fit completely in main memory and 2) efficient retrieval in low-resource environments (i.e., on devices with low computational resources, memory, or disk).

Acknowledgements

This work was partially supported by the Horizon Europe RIA “Extreme Food Risk Analytics” (EFRA), grant agreement n. 101093026, by the PNRR - M4C2 - Investimento 1.3, Partenariato Esteso PE00000013 - “FAIR - Future Artificial Intelligence Research” - Spoke 1 “Human-centered AI” funded by the European Commission under the NextGeneration EU program, by the PNRR ECS00000017 Tuscany Health Ecosystem Spoke 6 “Precision medicine & personalized healthcare” funded by the European Commission under the NextGeneration EU programme, by the MUR-PRIN 2017 “Algorithms, Data Structures and Combinatorics for Machine Learning”, grant agreement n. 2017K7XPAN_003, and by the MUR-PRIN 2022 “Algorithmic Problems and Machine Learning”, grant agreement n. 20229BCXNW.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Altingovde, I.S., Demir, E., Can, F., Ulusoy, O.: Incremental cluster-based retrieval using compressed cluster-skipping inverted files. *ACM Transactions on Information Systems* **26**(3) (Jun 2008)
2. Bruch, S., Gai, S., Ingber, A.: An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems* **42**(1) (August 2023)
3. Bruch, S., Lucchese, C., Nardini, F.M.: Efficient and effective tree-based and neural learning to rank. *Foundations and Trends in Information Retrieval* **17**(1), 1–123 (2023)
4. Bruch, S., Nardini, F.M., Ingber, A., Liberty, E.: An approximate algorithm for maximum inner product search over streaming sparse vectors. *ACM Transactions on Information Systems* **42**(2) (November 2023)
5. Bruch, S., Nardini, F.M., Ingber, A., Liberty, E.: Bridging dense and sparse maximum inner product search. *ACM Transactions on Information Systems* **42**(6) (Aug 2024)

6. Bruch, S., Nardini, F.M., Rulli, C., Venturini, R.: Efficient inverted indexes for approximate retrieval over learned sparse representations. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 152–162 (2024)
7. Bruch, S., Nardini, F.M., Rulli, C., Venturini, R.: Pairing clustered inverted indexes with κ -nn graphs for fast approximate retrieval over learned sparse representations. In: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. pp. 3642–3646 (2024)
8. Cinar, E.R., Altingovde, I.S.: Exploiting cluster-skipping inverted index for semantic place retrieval. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 1981–1985 (2023)
9. Delfino, L., Erriquez, D., Martinico, S., Nardini, F.M., Rulli, C., Venturini, R.: kANNolo: Sweet and smooth approximate k-nearest neighbors search (2025)
10. Formal, T., Lassance, C., Piwowarski, B., Clinchant, S.: From distillation to hard negative sampling: Making sparse neural ir models more effective. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 2353–2359 (2022)
11. Formal, T., Lassance, C., Piwowarski, B., Clinchant, S.: Towards effective and efficient sparse neural information retrieval. *ACM Transactions on Information Systems* (December 2023)
12. Formal, T., Lassance, C., Piwowarski, B., Clinchant, S.: Splade v2: Sparse lexical and expansion model for information retrieval (2021)
13. Formal, T., Piwowarski, B., Clinchant, S.: Splade: Sparse lexical and expansion model for first stage ranking. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 2288–2292 (2021)
14. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. pp. 6769–6781 (Nov 2020)
15. Khattab, O., Zaharia, M.: Colbert: Efficient and effective passage search via contextualized late interaction over bert. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 39–48 (2020)
16. Lassance, C., Clinchant, S.: An efficiency study for splade models. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 2220–2226 (2022)
17. Lassance, C., Lupart, S., Déjean, H., Clinchant, S., Tonellotto, N.: A static pruning study on sparse neural retrievers. In: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 1771–1775 (2023)
18. Lin, J., Nogueira, R.F., Yates, A.: *Pretrained Transformers for Text Ranking: BERT and Beyond. Synthesis Lectures on Human Language Technologies*, Morgan & Claypool Publishers (2021)
19. MacAvaney, S., Nardini, F.M., Perego, R., Tonellotto, N., Goharian, N., Frieder, O.: Expansion via prediction of importance with contextualization. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 1573–1576 (2020)
20. Mackenzie, J., Trotman, A., Lin, J.: Wacky weights in learned sparse representations and the revenge of score-at-a-time query evaluation (2021)

21. Mackenzie, J., Trotman, A., Lin, J.: Efficient document-at-a-time and score-at-a-time query evaluation for learned sparse representations. *ACM Transactions on Information Systems* **41**(4) (2023)
22. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **42**(4), 824–836 (4 2020)
23. Mallia, A., Mackenzie, J., Suel, T., Tonello, N.: Faster learned sparse retrieval with guided traversal. In: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. pp. 1901–1905 (2022)
24. Nardini, F.M., Rulli, C., Venturini, R.: Efficient multi-vector dense retrieval with bit vectors. In: *Advances in Information Retrieval*. pp. 3–17 (2024)
25. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: Ms marco: A human generated machine reading comprehension dataset (November 2016)
26. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics (11 2019)
27. Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., Zaharia, M.: ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pp. 3715–3734 (Jul 2022)
28. Simhadri, H.V., Aumüller, M., Ingber, A., Douze, M., Williams, G., Manohar, M.D., Baranchuk, D., Liberty, E., Liu, F., Landrum, B., Karjekar, M., Dhulipala, L., Chen, M., Chen, Y., Ma, R., Zhang, K., Cai, Y., Shi, J., Chen, Y., Zheng, W., Wan, Z., Yin, J., Huang, B.: Results of the big ann: Neurips’23 competition (2024)
29. Thakur, N., Wang, K., Gurevych, I., Lin, J.: Sprint: A unified toolkit for evaluating and demystifying zero-shot neural sparse retrieval. In: *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. pp. 2964–2974 (2023)
30. Tonello, N., Macdonald, C., Ounis, I.: Efficient query processing for scalable web search. *Foundations and Trends in Information Retrieval* **12**(4–5), 319–500 (December 2018)
31. Xiong, L., Xiong, C., Li, Y., Tang, K.F., Liu, J., Bennett, P., Ahmed, J., Overwijk, A.: Approximate nearest neighbor negative contrastive learning for dense text retrieval. In: *International Conference on Learning Representations* (4 2021)